

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

Дисциплина «Информационные технологии и программирование»

Отчет по лабораторной работе №2
«Переменные и арифметические операции»

Выполнил студент группы
ПИЖ-б-о-26-2
Панюков Алексей Юрьевич
Проверила
К.т.н., доцент Николаев Е.И.

г. Ставрополь, 2026 г.

Цель: Научиться объявлять переменные базовых типов (int, double, float, char, bool), выполнять арифметические операции (+, -, *, /, %), правильно выбирать тип данных в зависимости от решаемой задачи, понимать механизмы явного и неявного приведения типов, избегать потери точности при вычислениях, корректно работать с целочисленным и вещественным делением.

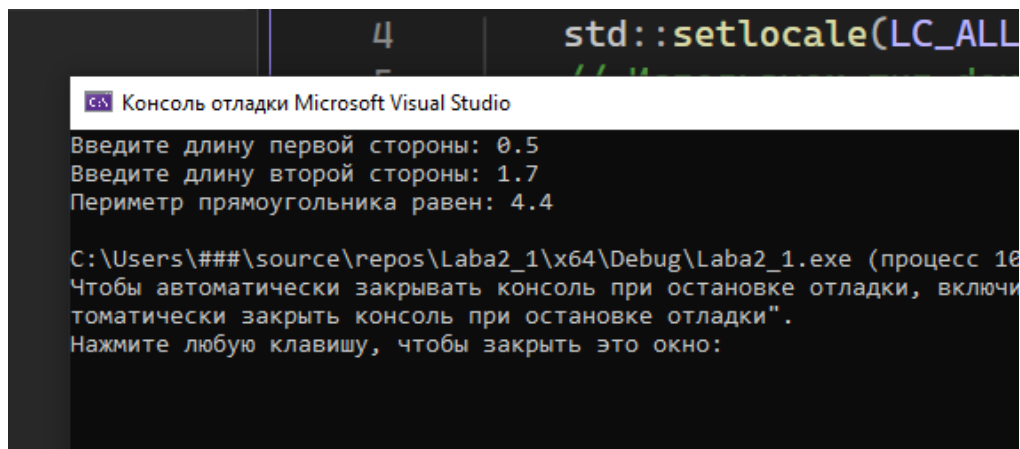
Ход работы

ЗАДАНИЕ 1

(Вычислить периметр прямоугольника со сторонами a и b)

Рис 1.1. – Готовый код

1. Введем корректные данные.



```
4 std::setlocale(LC_ALL, "ru");
5
Консоль отладки Microsoft Visual Studio
Введите длину первой стороны: 0.5
Введите длину второй стороны: 1.7
Периметр прямоугольника равен: 4.4

C:\Users\###\source\repos\Laba2_1\x64\Debug\Laba2_1.exe (процесс 16
Чтобы автоматически закрывать консоль при остановке отладки, включи
томатически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

Рис 1.2. – Результат с корректными данными

2. Введем некорректные данные.

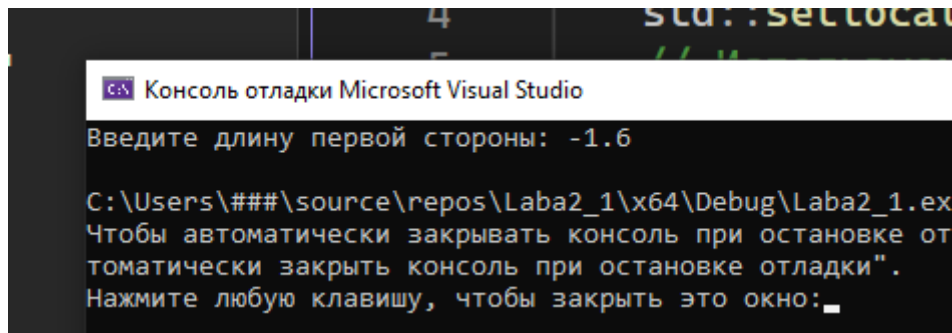


Рис 1.3. – Результат с некорректными данными

3. Введем некорректные данные другого типа.

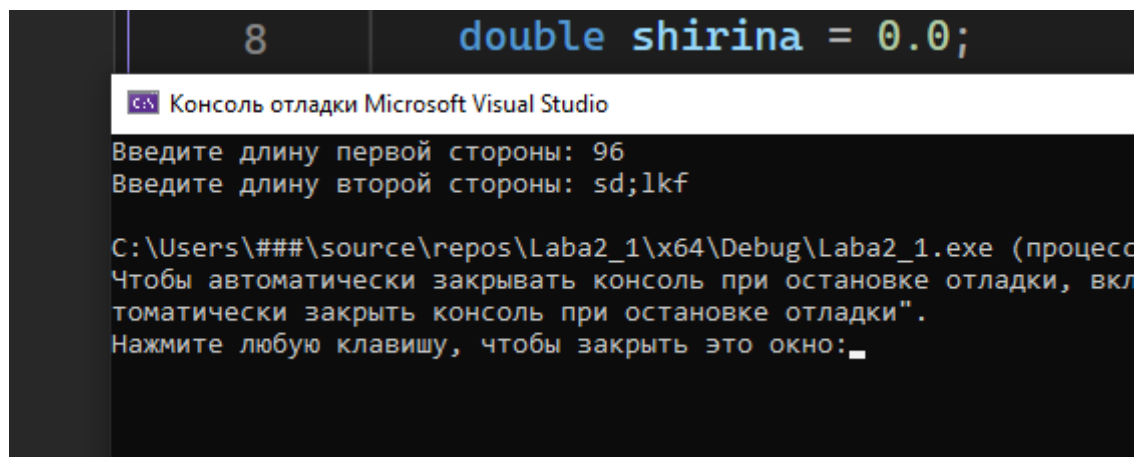


Рис 1.4. – Результат с некорректными данными

Листинг кода:

```
#include <iostream>

int main() {
    std::setlocale(LC_ALL, "Russian");
    // Используем тип double, так как стороны могут быть дробными.
    // Тип int подходит только для целых чисел, а double позволяет хранить числа с
    плавающей точкой.
    double dlina = 0.0;
    double shirina = 0.0;

    std::cout << "Введите длину первой стороны: ";
    std::cin >> dlina;
    if (dlina < 0 || std::cin.fail()) { //
        Проверка корректности
        return 0;
    }

    std::cout << "Введите длину второй стороны: ";
    std::cin >> shirina;
    if (shirina < 0 || std::cin.fail()) { //
        Проверка корректности
        return 0;
    }
}
```

```

}

// Вычисляем периметр по формуле:  $P = 2 * (a + b)$ 
double perimeter = 2.0 * (dlina + shirina);

// Выводим результат на экран
std::cout << "Периметр прямоугольника равен: " << perimeter << std::endl;

return 0;
}

```

ЗАДАНИЕ 2

(Вычислить среднее арифметическое и среднее геометрическое)

1. Введем корректные данные

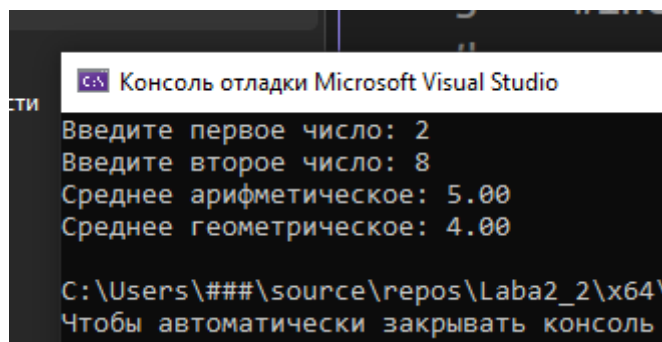


Рис 2.1 – Результат с корректными данными.

2. Введем некорректные данные.

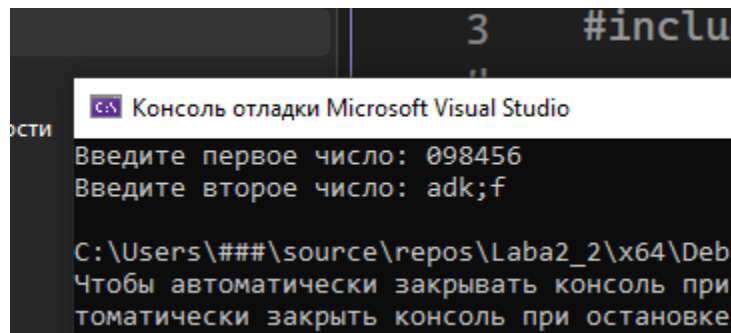


Рис 2.2. – Результат с некорректными данными

3. Введем другой тип некорректных данных

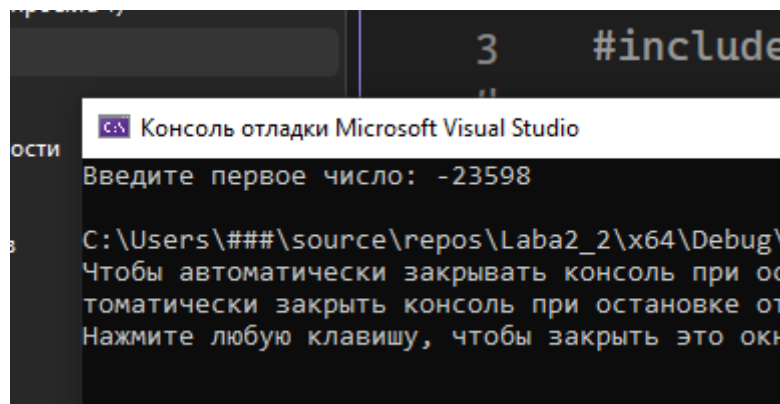


Рис 2.3. – Результат с некорректными данными

Листинг кода:

```
#include <iostream>
#include <cmath>
#include <iomanip>

int main() {
    std::setlocale(LC_ALL, "Russian");
    std::cout << std::fixed << std::setprecision(2);

    double first = 0.0, second = 0.0;    // исходные числа

    std::cout << "Введите первое число: ";
    std::cin >> first;
    if (first < 0 || std::cin.fail()) {    //
        Проверка корректности
        return 0;
    }

    std::cout << "Введите второе число: ";
    std::cin >> second;
```

```

    if (second < 0 || std::cin.fail()) {                                     //
        Проверка корректности
        return 0;
    }

    double arifm = static_cast<double>(first + second) / 2.0;             // явное
    среднее арифметическое
    double geomet = std::sqrt(static_cast<double>(first * second));        //
    вычисляем среднее геометрическое

    std::cout << "Среднее арифметическое: " << arifm << std::endl;
    std::cout << "Среднее геометрическое: " << geomet << std::endl;

    return 0;
}

```

Ответы на контрольные вопросы:

1. Приведение типов -преобразование значения одного типа в другой.
2. Явное указывает программист (`static_cast<int>(x)`), неявное компилятор автоматически.
3. При присваивании, в арифметических выражениях со смешанными типами, при передаче аргументов в функции.
4. Оба операнда `int`, выполняется целочисленное деление $5/2=2$, затем результат приводится к `double = 2.0`.
5. `double result = static_cast<double>(5) / 2;`
6. `static_cast` оператор явного приведения C++, проверяется компилятором, безопаснее и читаемее C-style.
7. Дробная часть отбрасывается, $x = 3$. Потеряются данные после запятой.
8. `%` определён только для целых типов. Обход: `static_cast<int> % static_cast<int>` или `std::fmod`.
9. Усечение - отбрасывание дробной части. Округление - математическое .
10. Потеря точности - утрата знаков при сужении типа. `double d = 1.23456789012345;` `float f = d;` -float хранит ~7 знаков.
11. Переполнение -выход за диапазон типа. `INT_MAX + 1 = INT_MIN`
12. Базовые понятия: типы данных определяют размер, диапазон и операции; переменные -именованные области памяти.
13. Переменная -именованная область памяти для хранения значения. Этапы: написание кода = компиляция = компоновка = исполнение.

14. Правила: буквы, цифры, `_`, не начинается с цифры, не ключевое слово.

Правильно: `studentCount`, `price_per_kg`. Неправильно: `2var`, `int`, `my-var`.

15.

Тип	Размер	Диапазон	Когда использовать
Int	4	$\pm 2 \cdot 10^9$	целые числа
double	8	$\pm 1.7 \cdot 10^{308}$	вещественные вычисления
float	4	$\pm 3.4 \cdot 10^{38}$	экономия памяти
char	1	-128...127	символы
bool	1	true/false	логические значения

16. `int` -знаковый ($\pm$), `unsigned int` -только ≥ 0 , вдвое больший положительный диапазон. Использовать для счётчиков, размеров, где отрицательные невозможны.

17. Студенты -`int`; цена -`double`; рост -`double`; сумма транзакции -`long` `long` или `double` (для точности -целые копейки).

18. Приведение типов -преобразование одного типа в другой. Явное задаёт программист, неявное выполняет компилятор.

19. `int a = 3.14; = 3 ; double b = 5; = 5.0 ; char c = 65; = 'A'.`

20. `static_cast` -оператор явного приведения. Лучше C-style: проверяется компилятором, виден в коде, безопаснее. Пример: `static_cast<double>(a) / b`.

21. 5 и 2 -`int` = целочисленное деление $5/2=2$ = приведение к `double` = 2.0.

22. Три способа: `static_cast<double>(a)/b`, `a/static_cast<double>(b)`, `a/2.0`.

23. `int a = 3.14; = a=3; double b = 5; = b=5.0; char c = 65; = c='A'; int d = c; = d=65.`

24. Пояснения: усечение, безопасное расширение, `int=char` по ASCII, `char=int` по коду.

25. Потеря точности -утрата знаков. `float f = 1.0f/3.0f`; даёт `~0.333333`, `double` - `~0.3333333333333333`.
26. Усечение -отбрасывание. Округление -`std::round 3.9=4`. В C++: `#include <cmath>`, `std::round()`.
27. Переполнение -выход за диапазон. `INT_MAX + 1 = INT_MIN`.
28. `%` работает только с целыми. Для `double`: `std::fmod`.
29. Операции: `+` `-` `*` `/` `%`. Приоритет: `*` `/` `%` выше `+` `-`. Ассоциативность слева направо.
30. Для целых `/` -целочисленное `7/2=3`. Для вещественных -обычное `7.0/2=3.5`.
31. Выведет 3. `x/y` -целочисленное деление `10/3=3`, затем `= 3.0`.
32. `double avg = (a + b + c) / 3.0;`
33. Магические числа -числа без пояснения в коде. Заменять на `const`, чтобы код был понятным и легко изменяемым.
34. Раздел Г: ввод/вывод и форматирование.
35. `std::cout` -вывод, `std::cin` -ввод. Операторы: `<<` для вывода, `>>` для ввода.
36. `std::endl` -перевод строки + сброс буфера; `\n` -только перевод строки. `\n` быстрее, `std::endl` -при необходимости сброса.
37. `#include <iomanip>`, затем `std::cout << std::fixed << std::setprecision(3) << value;`
38. `std::fixed` -фиксированный формат, `std::setprecision(n)` -n знаков после запятой.
39. `std::cout << "a = " << a << ", b = " << b << std::endl;`
40. Ошибки: нет `int` перед `main`, нет `std::` перед `cout/cin`, `cin >> a, b`; -нужно `cin >> a >> b`;; `a/b` -целочисленное деление, "Результат: " + `c` -нельзя складывать строку и число.
41. Выведет "Не равны" `-0.1+0.2 ≠ 0.3` из-за погрешности представления `double`.
42. `Double` хранит числа приближённо. Сравнивать через `std::abs(a - b) < epsilon`.
43. `3.14` -`double`; `3.14f` -`float`; `42L` -`long`; `42U` -`unsigned int`.

44. `const` -неизменяемое значение. Нужны для защиты от изменений и читаемости. Пример: `const double PI = 3.14159;`

45. `static_assert` -проверка условия на этапе компиляции. Помогает убедиться, что тип имеет нужный размер, например `static_assert(sizeof(int) == 4);`

Ссылка на созданный репозиторий:

(https://github.com/Duster223/Vvedenie_V_Programmirovaniye)

Вывод: В ходе лабораторной работы были освоены объявление переменных базовых типов (`int`, `double`, `char`, `bool`), выполнение арифметических операций (+, -, *, /, %) и выбор типа данных в зависимости от решаемой задачи. Была продемонстрирована разница между целочисленным и вещественным делением, применено явное приведение типов через `static_cast` и форматирование вывода с помощью `std::fixed` и `std::setprecision`. Реализованы корректный ввод/вывод данных через `std::cin` и `std::cout`, и осмысленные имена переменных. Программы компилируются и работают без ошибок.